

TUGAS BESAR IF3170
TEORI BAHASA FORMAL DAN OTOMATA
MILESTONE 2 - SYNTAX ANALYSIS



Muhammad Ra'if Alkautsar	13523011
Fajar Kurniawan	13523076
Darrel Adinarya Sunanda	13523061
Reza Ahmad Syarif	13523119

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESHA 10, BANDUNG 40132
2025

Daftar Isi

Daftar Isi.....	2
1. Landasan Teori.....	3
a. Grammar.....	3
b. Parse Tree.....	3
c. Syntax Analysis (Parser).....	3
d. Recursive Descent.....	4
2. Perancangan & Implementasi Program.....	6
a. Teknologi Yang Digunakan.....	6
b. Struktur Program.....	6
c. Fungsi/Kelas Utama.....	7
d. Alur Kerja Program.....	9
3. Pengujian.....	10
a. dasar.pas.....	10
b. medium.pas.....	13
c. complete.pas.....	27
d. hard.pas.....	31
e. error_test.pas.....	35
4. Kesimpulan & Saran.....	39
a. Kesimpulan.....	39
b. Saran.....	39
Lampiran.....	40
Referensi.....	43

1. Landasan Teori

Dalam pemrosesan bahasa pemrograman, setelah tahap analisis leksikal (*lexical analysis*), terhadap tahap analisis sintaksis (*syntax analysis*) oleh parser. Token yang dihasilkan oleh tahap lexical analysis digunakan di tahap *syntax analysis* ini untuk di-*parsing* menjadi sebuah parse tree. Parse tree yang dihasilkan ini akan digunakan untuk tahap yang selanjutnya.

a. Grammar

Secara *high level*, sebuah grammar adalah sekumpulan aturan yang menjelaskan struktur sebuah bahasa. Aturan-aturan tersebut mendefinisikan string apa saja yang valid di dalam bahasa tersebut. Bagian-bagian dari sebuah grammar adalah sebagai berikut.

- Simbol Terminal – Simbol dasar yang membentuk suatu bahasa. Mereka bisa dalam bentuk huruf, angka, dan operator seperti $\{0,1\}$, $\{0,1\}$.
- Simbol Nonterminal – Simbol yang merepresentasikan sekelompok simbol terminal atau nonterminal yang lain. Mereka digunakan untuk menyusun struktur dari aturan-aturan sebuah grammar.
- Produksi/*Production* – Aturan yang mendeskripsikan bagaimana non-terminal bisa diganti dengan sebuah terminal atau nonterminal yang lain.
- Starting Nonterminal (Start Symbol) – Sebuah nonterminal khusus yang merupakan awal dari penurunan/*derivation*.

Sebagai contoh, di sebuah bahasa pemrograman, terminal simbol bisa berupa +, -, a, b, atau angka. Non-terminal bisa berupa sebuah ekspresi atau statement.

b. Parse Tree

Parse Tree adalah representasi lengkap dari struktur sintaks program sesuai dengan grammar bahasa yang digunakan. Setiap node di dalam parse tree merepresentasikan produksi grammar, sehingga pohon ini memuat semua simbol terminal dan non-terminal. Parse tree berguna untuk menunjukkan bagaimana token-token hasil lexical analysis membentuk struktur program yang sesuai dengan aturan grammar.

Proses parsing dimulai dari start symbol yang merupakan root dari parse tree. Parse tree yang sudah lengkap mengikuti operator precedence dan subpohon yang lebih dalam akan dieksekusi terlebih dahulu. Ini berarti sebuah node akan dieksekusi sebelum parentnya. Jika ada aturan produksi $A \rightarrow C_1, C_2, \dots, C_n$ maka $C_1, C_2, \dots, C_n$ adalah anak dari simpul yang berlabel A. Simpul daun pada parse tree merepresentasikan terminal (Σ) sedangkan simpul dalam/interior merepresentasikan variabel (V).

c. Syntax Analysis (Parser)

Syntax Analysis atau Parser bertugas untuk memeriksa urutan token yang dihasilkan oleh lexical analyzer agar sesuai dengan aturan tata bahasa (grammar)

dari bahasa pemrograman yang dikompilasi. Parser akan membangun struktur sintaks seperti Parse Tree yang merepresentasikan struktur hierarkis program sumber berdasarkan grammar yang telah didefinisikan.

Tahapan Parser bertujuan untuk:

- Memastikan urutan token membentuk struktur sintaks yang valid.
- Mendeteksi dan melaporkan kesalahan sintaks (syntax error).
- Mempersiapkan representasi program untuk tahap berikutnya.

d. Recursive Descent

Recursive Descent Parser adalah parser yang menggunakan teknik top-down parsing tanpa menggunakan backtracking. Ini dapat didefinisikan sebagai sebuah parser yang menggunakan berbagai prosedur rekursif untuk memproses string masukan tanpa menggunakan backtracking.

Aturan produksi pada grammar yang digunakan direpresentasikan dengan mengaitkan sebuah non-terminal di ruas kiri aturan produksi dengan sebuah prosedur. Prosedur tersebut jika dipanggil akan berusaha membaca serangkaian simbol yang bisa dihasilkan dari aturan produksi milik simbol non-terminal dari prosedur tersebut. Kemudian jika salah satu dari simbol yang dibaca tadi merupakan non-terminal maka prosedur terkait akan dipanggil. Setelah selesai dibaca maka dikembalikan pointer ke parent dari non-terminal yang sedang diproses sehingga setelah parse tree berhasil dibuat maka dikembalikan pointer ke root dari parse tree.

Recursive descent dapat memiliki sebuah variabel lookahead untuk menyimpan token yang sedang diproses. Kemudian ada sebuah prosedur match untuk mengecek apakah token selanjutnya adalah expected token untuk aturan produksi terkait sehingga mengubah value dari lookahead menjadi token selanjutnya. Implementasinya dapat bervariasi dan tidak harus menggunakan variabel lookahead serta prosedur match(), yang jelas parser harus dapat melihat token di depan current token untuk mengecek apakah match dengan simbol yang di-expect.

Contoh recursive descent parser adalah sebagai berikut:

Grammar:

$$E \rightarrow i E'$$
$$E' \rightarrow + i E' \mid \epsilon$$

Procedure E()

E()

{

 if (input == 'i') { // If the input is 'i' (identifier)

```
    input++;    // Consume 'i'  
  }  
  E();  // Call E' to check for further expressions  
}
```

2. Perancangan & Implementasi Program

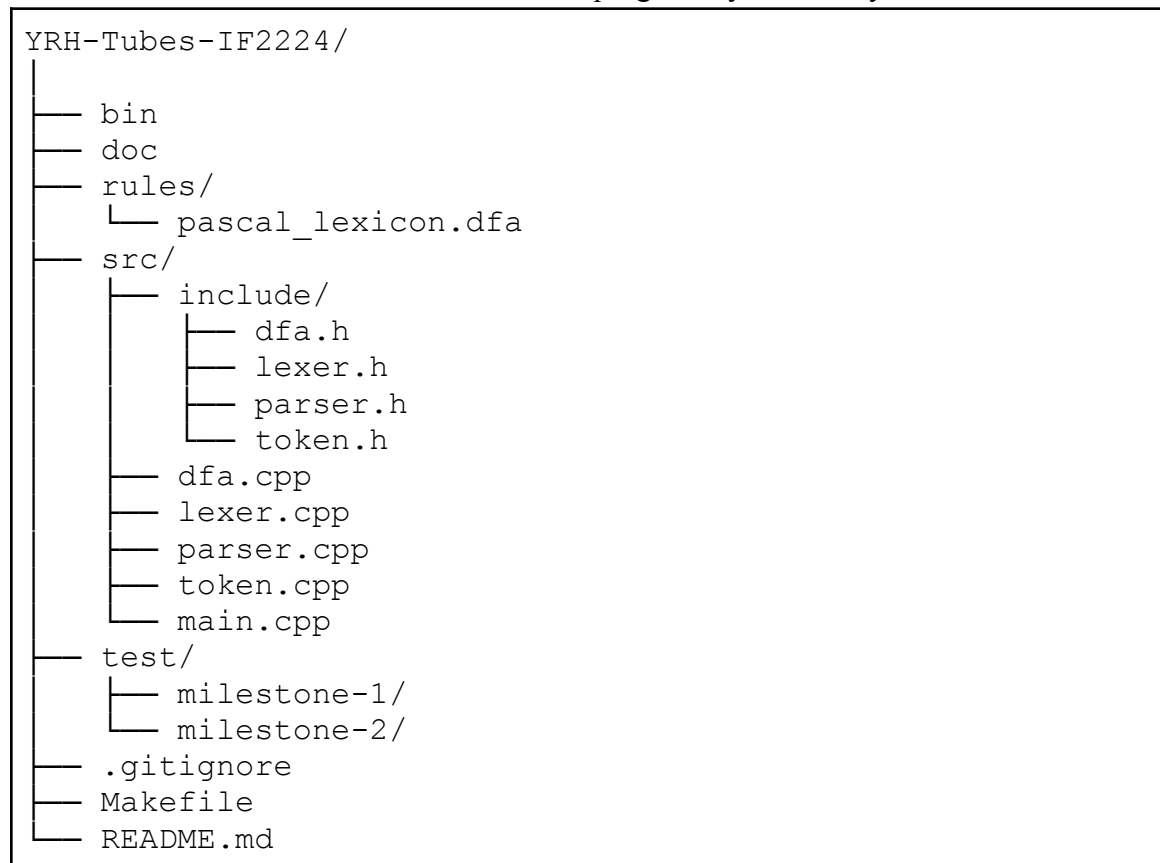
a. Teknologi Yang Digunakan

Implementasi syntax analyzer ini memanfaatkan dua teknologi dari C++17 yaitu *smart pointers* dan *standard library containers*. Smart pointers, khususnya `shared_ptr`, digunakan untuk manajemen memori otomatis pada struktur parse tree. Setiap `ParseNode` dalam tree menggunakan `shared_ptr` untuk menyimpan referensi ke *children nodes*. Pemilihan `shared_ptr` didasarkan pada kebutuhan *shared ownership* yaitu satu node dapat direferensikan oleh parent node, disimpan dalam collections, atau diakses dari berbagai bagian kode. Smart pointer menyediakan *reference counting* otomatis yang memastikan node akan di-dealokasi hanya ketika tidak ada lagi referensi yang menggunakannya sehingga dapat mengeliminasi risiko memory leak dan dangling pointer.

Standard library containers berupa `std::vector` digunakan dalam dua konteks utama yaitu `vector<Token*>` untuk menyimpan input token list dari lexer dan `vector<shared_ptr<ParseNode>>` untuk menyimpan children list dari setiap node. Vector dipilih karena menyediakan $O(1)$ random access yang merupakan aspek krusial untuk operasi lookahead dalam parser yang perlu "mengintip" token berikutnya tanpa mengkonsumsinya.

b. Struktur Program

Berikut adalah struktur direktori dan file dari program Syntax Analyzer ini.



Berikut adalah penjelasan dari struktur tersebut.

1. Header Files (include)
 - a. token.h: Mendefinisikan enum Type dan class Token untuk representasi token
 - b. dfa.h: Mendefinisikan DFA untuk pattern matching dalam lexer
 - c. lexer.h: Mendefinisikan Lexer class untuk lexical analysis
 - d. parser.h: Mendefinisikan Parser dan ParseNode class untuk syntax analysis
2. Implementation Files (src)
 - a. token.cpp: Implementasi Token class
 - b. dfa.cpp: Implementasi DFA state machine
 - c. lexer.cpp: Implementasi tokenization process
 - d. parser.cpp: Implementasi parsing dengan recursive descent
3. Resources
 - a. pascal_lexicon.dfa: definisi aturan DFA
 - b. milestone-1: Test cases untuk lexical analyzer
 - c. milestone-2: Test cases untuk syntax analyzer

c. Fungsi/Kelas Utama

Berikut adalah kelas utama dan beberapa fungsi didalamnya yang berperan pada proses syntax analysis.

1. Kelas ParseNode

Kelas ini berfungsi untuk merepresentasikan setiap node dalam parse tree, baik terminal maupun non-terminal. Kelas ini memiliki tiga atribut yaitu nodeType untuk menyimpan jenis node, value untuk menyimpan nilai token simbol terminal, dan children yang berupa vector yang berisi child nodes. Kelas ini mempunyai dua method utama yaitu addChild() untuk menambahkan child nodes ke children list dan printTree() untuk mencetak parse tree.

```
class ParseNode {
private:
    string nodeType;
    string value;
    vector<shared_ptr<ParseNode>> children;

public:
    ParseNode(const string& type, const string& val =
    "");
    void addChild(shared_ptr<ParseNode> child);
    void print(int indent = 0) const;
    void printTree(const string& prefix = "", bool isLast
    = true) const;
    string getType() const { return nodeType; }
    string getValue() const { return value; }
```

```

    const vector<shared_ptr<ParseNode>>& getChildren()
const { return children; }
};

```

2. Kelas Parser

Kelas ini berfungsi untuk melakukan analisis sintaks menggunakan recursive descent parsing dengan grammar Context-Free Grammar (CFG). Kelas ini memiliki tiga atribut yaitu tokens yang berupa vector berisi daftar token hasil lexical analysis, currentPos untuk menyimpan posisi token yang sedang diproses, dan errors untuk menyimpan vector untuk menyimpan syntax error yang ditemukan. Kelas ini mempunyai beberapa fungsi navigasi dan validasi token serta fungsi dengan nama parse__ yang merepresentasikan satu production rule dalam grammar.

```

class Parser {
private:
    vector<Token*> tokens;
    size_t currentPos;
    vector<string> errors;

    Token* currentToken();
    Token* peek(int offset = 1);
    void advance();
    bool match(Type type);
    bool matchKeyword(const string& keyword);
    void expect(Type type, const string& errorMsg);
    void expectKeyword(const string& keyword, const
string& errorMsg);
    void syntaxError(const string& message);

    shared_ptr<ParseNode> parseProgram();
    shared_ptr<ParseNode> parseProgramHeader();
    shared_ptr<ParseNode> parseDeclarationPart();
    shared_ptr<ParseNode> parseConstDeclaration();
    shared_ptr<ParseNode> parseTypeDeclaration();
    shared_ptr<ParseNode> parseTypeDefinition();
    shared_ptr<ParseNode> parseVarDeclaration();
    shared_ptr<ParseNode> parseIdentifierList();
    shared_ptr<ParseNode> parseType();
    shared_ptr<ParseNode> parseArrayType();
    shared_ptr<ParseNode> parseRecordType();
    shared_ptr<ParseNode> parseRange();
    shared_ptr<ParseNode> parseSubprogramDeclaration();
    shared_ptr<ParseNode> parseProcedureDeclaration();
    shared_ptr<ParseNode> parseFunctionDeclaration();
    shared_ptr<ParseNode> parseFormalParameterList();
    shared_ptr<ParseNode> parseCompoundStatement();
    shared_ptr<ParseNode> parseStatementList();

```



```

shared_ptr<ParseNode> parseAssignmentStatement();
shared_ptr<ParseNode> parseIfStatement();
shared_ptr<ParseNode> parseWhileStatement();
shared_ptr<ParseNode> parseForStatement();
shared_ptr<ParseNode> parseProcedureFunctionCall();
shared_ptr<ParseNode> parseParameterList();
shared_ptr<ParseNode> parseExpression();
shared_ptr<ParseNode> parseSimpleExpression();
shared_ptr<ParseNode> parseTerm();
shared_ptr<ParseNode> parseFactor();
shared_ptr<ParseNode> parseRelationalOperator();
shared_ptr<ParseNode> parseAdditiveOperator();
shared_ptr<ParseNode> parseMultiplicationOperator();

public:
    Parser(const vector<Token*>& tokenList);
    shared_ptr<ParseNode> parse();
    const vector<string>& getErrors() const { return
errors; }
    bool hasErrors() const { return !errors.empty(); }
};

```

d. Alur Kerja Program

Program syntax analysis dimulai ketika kelas Parser menerima input berupa *vector of tokens* dari hasil lexical analysis. Proses parsing dimulai dengan memanggil method `parse()` yang kemudian memanggil `parseProgram()` sebagai entry point untuk memulai analisis dari root grammar. Parsing dilakukan menggunakan teknik Recursive Descent Parsing yang bersifat *top-down*. Setiap fungsi parsing membuat object `ParseNode` untuk merepresentasikan simbol non-terminal, kemudian memanggil fungsi-fungsi parsing lainnya secara rekursif untuk mem-parse children nodes sesuai grammar rules. Navigasi token dilakukan melalui helper functions seperti `currentToken()` untuk mendapatkan token saat ini, `peek()` untuk lookahead, dan `advance()` untuk memajukan posisi. Validasi token menggunakan `expect()` yang akan memanggil `syntaxError()` jika token tidak sesuai dan mencatat error detail tanpa menghentikan parsing sepenuhnya.

Selama proses parsing, setiap token terminal dan non-terminal ditambahkan sebagai node ke dalam parse tree untuk membangun struktur hierarkis program. Parsing expression dilakukan secara bertingkat mengikuti operator precedence, dimulai dari relational operators, kemudian additive operators, multiplicative operators, hingga operands dasar seperti identifier dan literal. Setelah seluruh token selesai diproses, method `parse()` mengembalikan root node dari parse tree yang merepresentasikan struktur sintaks lengkap program. Jika ditemukan syntax error selama parsing, informasi error dicatat dalam list namun proses tetap dilanjutkan untuk mendeteksi error lainnya. Parse tree hasil akhir dapat ditampilkan dalam format hierarkis menggunakan method

printTree().

3. Pengujian

a. dasar.pas

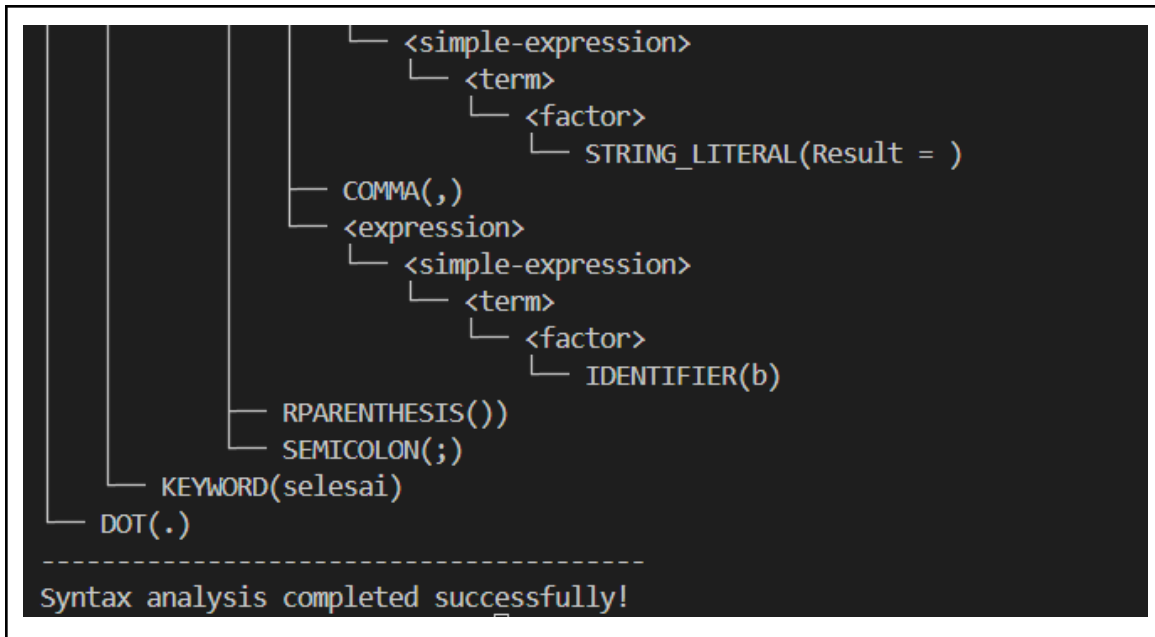
Input
<pre>program Hello; variabel a, b: integer; mulai a := 5; b := a + 10; writeln('Result = ', b); selesai.</pre>
Output
<pre>Using DFA-based lexer DFA rules file: rules/pascal_lexicon.dfa Processing file: test/milestone-2/dasar.pas ----- KEYWORD(program) IDENTIFIER(Hello) SEMICOLON(;) KEYWORD(variabel) IDENTIFIER(a) COMMA(,) IDENTIFIER(b) COLON(:) KEYWORD(integer) SEMICOLON(;) KEYWORD(mulai) IDENTIFIER(a) ASSIGN_OPERATOR(:=) NUMBER(5) SEMICOLON(;) IDENTIFIER(b) ASSIGN_OPERATOR(:=) IDENTIFIER(a) ARITHMETIC_OPERATOR(+) NUMBER(10) SEMICOLON(;) IDENTIFIER(writeln) LPARENTHESIS() STRING_LITERAL(Result =) COMMA(,) IDENTIFIER(b) RPARENTHESIS() SEMICOLON(;) KEYWORD(selesai) DOT(.)</pre>

Tokenization completed successfully!
Total tokens: 30

Starting syntax analysis...

Parse Tree:

```
<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER>Hello)
│   └── SEMICOLON(;)
├── <declaration-part>
│   └── <var-declaration>
│       ├── KEYWORD(variabel)
│       ├── <identifier-list>
│       │   ├── IDENTIFIER(a)
│       │   ├── COMMA(,)
│       │   └── IDENTIFIER(b)
│       ├── COLON(:)
│       ├── <type>
│       │   └── KEYWORD(integer)
│       └── SEMICOLON(;)
└── <compound-statement>
    ├── KEYWORD(mulai)
    ├── <statement-list>
    │   ├── <assignment-statement>
    │   │   ├── IDENTIFIER(a)
    │   │   ├── ASSIGN_OPERATOR(=)
    │   │   └── <expression>
    │   │       └── <simple-expression>
    │   │           └── <term>
    │   │               └── <factor>
    │   │                   └── NUMBER(5)
    │   ├── SEMICOLON(;)
    │   ├── <assignment-statement>
    │   │   ├── IDENTIFIER(b)
    │   │   ├── ASSIGN_OPERATOR(=)
    │   │   └── <expression>
    │   │       └── <simple-expression>
    │   │           ├── <term>
    │   │           │   ├── <factor>
    │   │           │   └── IDENTIFIER(a)
    │   │           ├── ARITHMETIC_OPERATOR(+)
    │   │           └── <term>
    │   │               └── <factor>
    │   │                   └── NUMBER(10)
    │   └── SEMICOLON(;)
    └── <procedure-call>
        ├── IDENTIFIER(writeln)
        ├── LPARENTHESIS( )
        └── <parameter-list>
            └── <expression>
                └── <simple-expression>
```

b. medium.pas

Input

```

program TestLexerIndonesiaLengkap;

{ =====
  Test Suite Lexical Analyzer - Milestone 2
  Semua Keyword Bahasa Indonesia
  ===== }

konstanta
  MAX_SIZE = 100;
  MIN_VALUE = 0;
  PI = 3.14159;
  NAMA_APP = 'Aplikasi Test';

tipe
  NilaiUjian = larik [1..10] dari integer;

variabel
  i, j, k: integer;
  hasil, total: real;
  selesai_flag: boolean;
  huruf: char;
  angka: NilaiUjian;
  mahasiswa: DataMahasiswa;
  status: StatusSiswa;

fungsi tambah(a, b: integer): integer;
mulai
  tambah := a + b;
selesai;

```

```

fungsi rata_rata(nilai: NilaiUjian; n: integer): real;
variabel
    idx: integer;
    jumlah: real;
mulai
    jumlah := 0;
    untuk idx := 1 ke n lakukan
        jumlah := jumlah + nilai[idx];
    rata_rata := jumlah bagi n;
selesai;

prosedur cetak_info(npm: integer; nama: char);
mulai
    { Cetak informasi mahasiswa }
    (* Format: NPM - Nama *)
selesai;

prosedur inisialisasi_array;
variabel
    idx: integer;
mulai
    untuk idx := 1 ke 10 lakukan
        angka[idx] := 0;
selesai;

mulai
    { ===== TEST CONDITIONAL (JIKA-MAKA-SELAIN-ITU) ===== }
    jika i > j maka
        mulai
            i := i + 1;
            j := j - 1;
        selesai
    selain_itu
        mulai
            i := i - 1;
            j := j + 1;
        selesai;
selesai.

```

Output

```

Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: test/milestone-2/medium.pas
-----
KEYWORD(program)
IDENTIFIER(TestLexerIndonesiaLengkap)
SEMICOLON(;)
KEYWORD(konstanta)
IDENTIFIER(MAX_SIZE)
RELATIONAL_OPERATOR(=)
NUMBER(100)
SEMICOLON(;)

```

```
IDENTIFIER(MIN_VALUE)
RELATIONAL_OPERATOR(=)
NUMBER(0)
SEMICOLON(;)
IDENTIFIER(PI)
RELATIONAL_OPERATOR(=)
NUMBER(3)
DOT(.)
NUMBER(14159)
SEMICOLON(;)
IDENTIFIER(NAMA_APP)
RELATIONAL_OPERATOR(=)
STRING_LITERAL(Aplikasi Test)
SEMICOLON(;)
KEYWORD(tipe)
IDENTIFIER(NilaiUjian)
RELATIONAL_OPERATOR(=)
KEYWORD(larik)
LBRACKET([)
NUMBER(1)
RANGE_OPERATOR(..)
NUMBER(10)
RBRACKET(])
KEYWORD(dari)
KEYWORD(integer)
SEMICOLON(;)
KEYWORD(variabel)
IDENTIFIER(i)
COMMA(,)
IDENTIFIER(j)
COMMA(,)
IDENTIFIER(k)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
IDENTIFIER(hasil)
COMMA(,)
IDENTIFIER(total)
COLON(:)
KEYWORD(real)
SEMICOLON(;)
IDENTIFIER(selesai_flag)
COLON(:)
KEYWORD(boolean)
SEMICOLON(;)
IDENTIFIER(huruf)
COLON(:)
KEYWORD(char)
SEMICOLON(;)
IDENTIFIER(angka)
COLON(:)
IDENTIFIER(NilaiUjian)
SEMICOLON(;)
IDENTIFIER(mahasiswa)
COLON(:)
IDENTIFIER(DataMahasiswa)
```

```
SEMICOLON (;)
IDENTIFIER (status)
COLON (:)
IDENTIFIER (StatusSiswa)
SEMICOLON (;)
KEYWORD (fungsi)
IDENTIFIER (tambah)
LPARENTHESIS ( ( )
IDENTIFIER (a)
COMMA (,)
IDENTIFIER (b)
COLON (:)
KEYWORD (integer)
RPARENTHESIS ( ) )
COLON (:)
KEYWORD (integer)
SEMICOLON (;)
KEYWORD (mulai)
IDENTIFIER (tambah)
ASSIGN_OPERATOR (:=)
IDENTIFIER (a)
ARITHMETIC_OPERATOR (+)
IDENTIFIER (b)
SEMICOLON (;)
KEYWORD (selesai)
SEMICOLON (;)
KEYWORD (fungsi)
IDENTIFIER (rata_rata)
LPARENTHESIS ( ( )
IDENTIFIER (nilai)
COLON (:)
IDENTIFIER (NilaiUjian)
SEMICOLON (;)
IDENTIFIER (n)
COLON (:)
KEYWORD (integer)
RPARENTHESIS ( ) )
COLON (:)
KEYWORD (real)
SEMICOLON (;)
KEYWORD (variabel)
IDENTIFIER (idx)
COLON (:)
KEYWORD (integer)
SEMICOLON (;)
IDENTIFIER (jumlah)
COLON (:)
KEYWORD (real)
SEMICOLON (;)
KEYWORD (mulai)
IDENTIFIER (jumlah)
ASSIGN_OPERATOR (:=)
NUMBER (0)
SEMICOLON (;)
KEYWORD (untuk)
IDENTIFIER (idx)
```



```
ASSIGN_OPERATOR(:=)
NUMBER(1)
KEYWORD(ke)
IDENTIFIER(n)
KEYWORD(lakukan)
IDENTIFIER(jumlah)
ASSIGN_OPERATOR(:=)
IDENTIFIER(jumlah)
ARITHMETIC_OPERATOR(+)
IDENTIFIER(nilai)
LBRACKET([)
IDENTIFIER(idx)
RBRACKET(])
SEMICOLON(;)
IDENTIFIER(rata_rata)
ASSIGN_OPERATOR(:=)
IDENTIFIER(jumlah)
ARITHMETIC_OPERATOR(bagi)
IDENTIFIER(n)
SEMICOLON(;)
KEYWORD(selesai)
SEMICOLON(;)
KEYWORD(prosedur)
IDENTIFIER(cetak_info)
LPARENTHESIS( )
IDENTIFIER(npm)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
IDENTIFIER(nama)
COLON(:)
KEYWORD(char)
RPARENTHESIS() )
SEMICOLON(;)
KEYWORD(mulai)
KEYWORD(selesai)
SEMICOLON(;)
KEYWORD(prosedur)
IDENTIFIER(inisialisasi_array)
SEMICOLON(;)
KEYWORD(variabel)
IDENTIFIER(idx)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
KEYWORD(mulai)
KEYWORD(untuk)
IDENTIFIER(idx)
ASSIGN_OPERATOR(:=)
NUMBER(1)
KEYWORD(ke)
NUMBER(10)
KEYWORD(lakukan)
IDENTIFIER(angka)
LBRACKET([)
IDENTIFIER(idx)
```

```

RBRACKET (])
ASSIGN_OPERATOR (:=)
NUMBER (0)
SEMICOLON (;)
KEYWORD (selesai)
SEMICOLON (;)
KEYWORD (mulai)
KEYWORD (jika)
IDENTIFIER (i)
RELATIONAL_OPERATOR (>)
IDENTIFIER (j)
KEYWORD (maka)
KEYWORD (mulai)
IDENTIFIER (i)
ASSIGN_OPERATOR (:=)
IDENTIFIER (i)
ARITHMETIC_OPERATOR (+)
NUMBER (1)
SEMICOLON (;)
IDENTIFIER (j)
ASSIGN_OPERATOR (:=)
IDENTIFIER (j)
ARITHMETIC_OPERATOR (-)
NUMBER (1)
SEMICOLON (;)
KEYWORD (selesai)
KEYWORD (selain itu)
KEYWORD (mulai)
IDENTIFIER (i)
ASSIGN_OPERATOR (:=)
IDENTIFIER (i)
ARITHMETIC_OPERATOR (-)
NUMBER (1)
SEMICOLON (;)
IDENTIFIER (j)
ASSIGN_OPERATOR (:=)
IDENTIFIER (j)
ARITHMETIC_OPERATOR (+)
NUMBER (1)
SEMICOLON (;)
KEYWORD (selesai)
SEMICOLON (;)
KEYWORD (selesai)
DOT (.)

-----

Tokenization completed successfully!
Total tokens: 220

-----

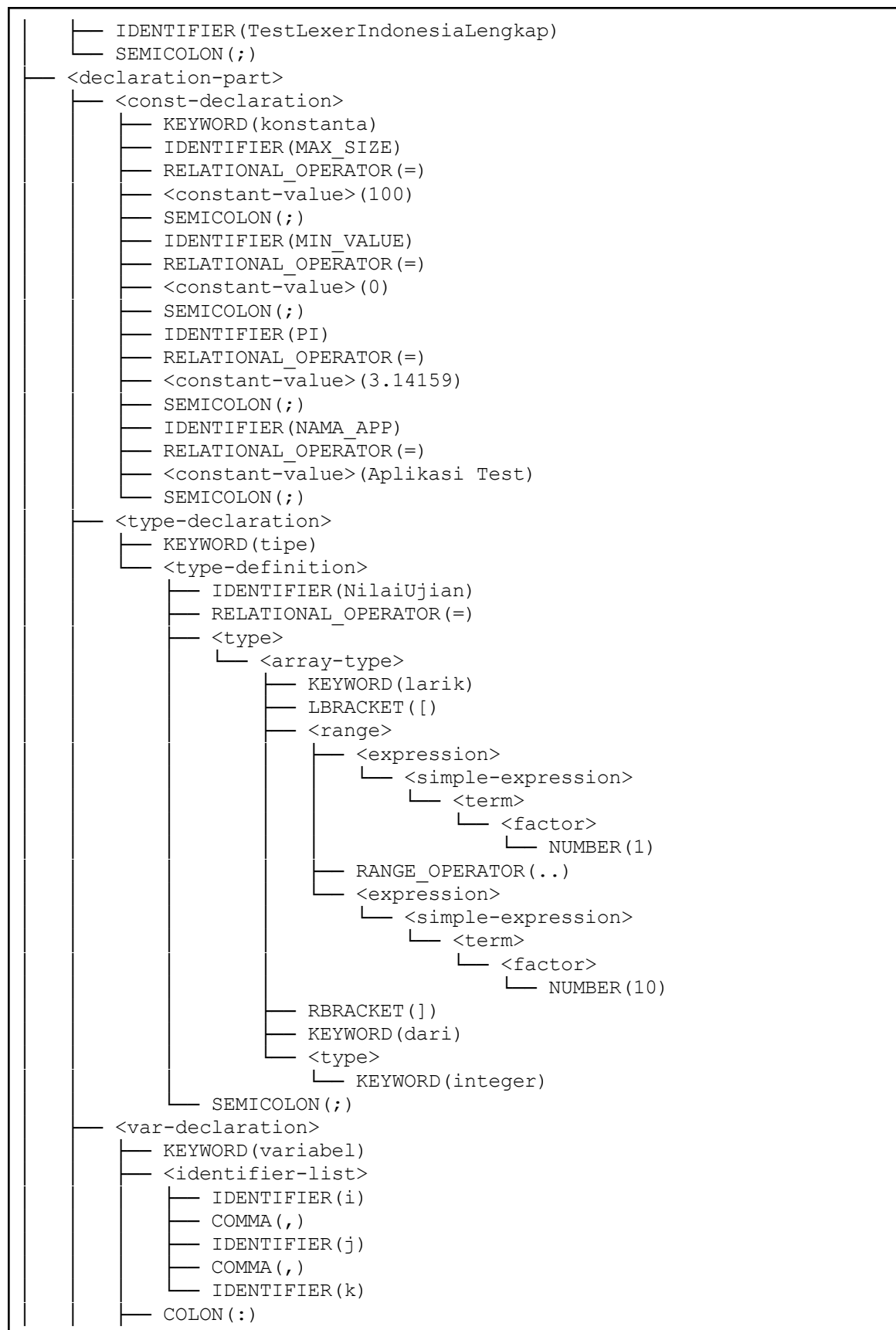
Starting syntax analysis...

-----

Parse Tree:

-----
<program>
├── <program-header>
│   └── KEYWORD (program)

```



```

— <type>
  — KEYWORD(integer)
— SEMICOLON(;)
— <identifier-list>
  — IDENTIFIER(hasil)
  — COMMA(,)
  — IDENTIFIER(total)
— COLON(:)
— <type>
  — KEYWORD(real)
— SEMICOLON(;)
— <identifier-list>
  — IDENTIFIER(selesai_flag)
— COLON(:)
— <type>
  — KEYWORD(boolean)
— SEMICOLON(;)
— <identifier-list>
  — IDENTIFIER(huruf)
— COLON(:)
— <type>
  — KEYWORD(char)
— SEMICOLON(;)
— <identifier-list>
  — IDENTIFIER(angka)
— COLON(:)
— <type>
  — <custom-type>(NilaiUjian)
— SEMICOLON(;)
— <identifier-list>
  — IDENTIFIER(mahasiswa)
— COLON(:)
— <type>
  — <custom-type>(DataMahasiswa)
— SEMICOLON(;)
— <identifier-list>
  — IDENTIFIER(status)
— COLON(:)
— <type>
  — <custom-type>(StatusSiswa)
— SEMICOLON(;)
— <subprogram-declaration>
  — <function-declaration>
    — KEYWORD(fungsi)
    — IDENTIFIER(tambah)
    — <formal-parameter-list>
      — LPARENTHESIS( )
      — <identifier-list>
        — IDENTIFIER(a)
        — COMMA(,)
        — IDENTIFIER(b)
      — COLON(:)
      — <type>
        — KEYWORD(integer)
      — RPARENTHESIS( )
    — COLON(:)

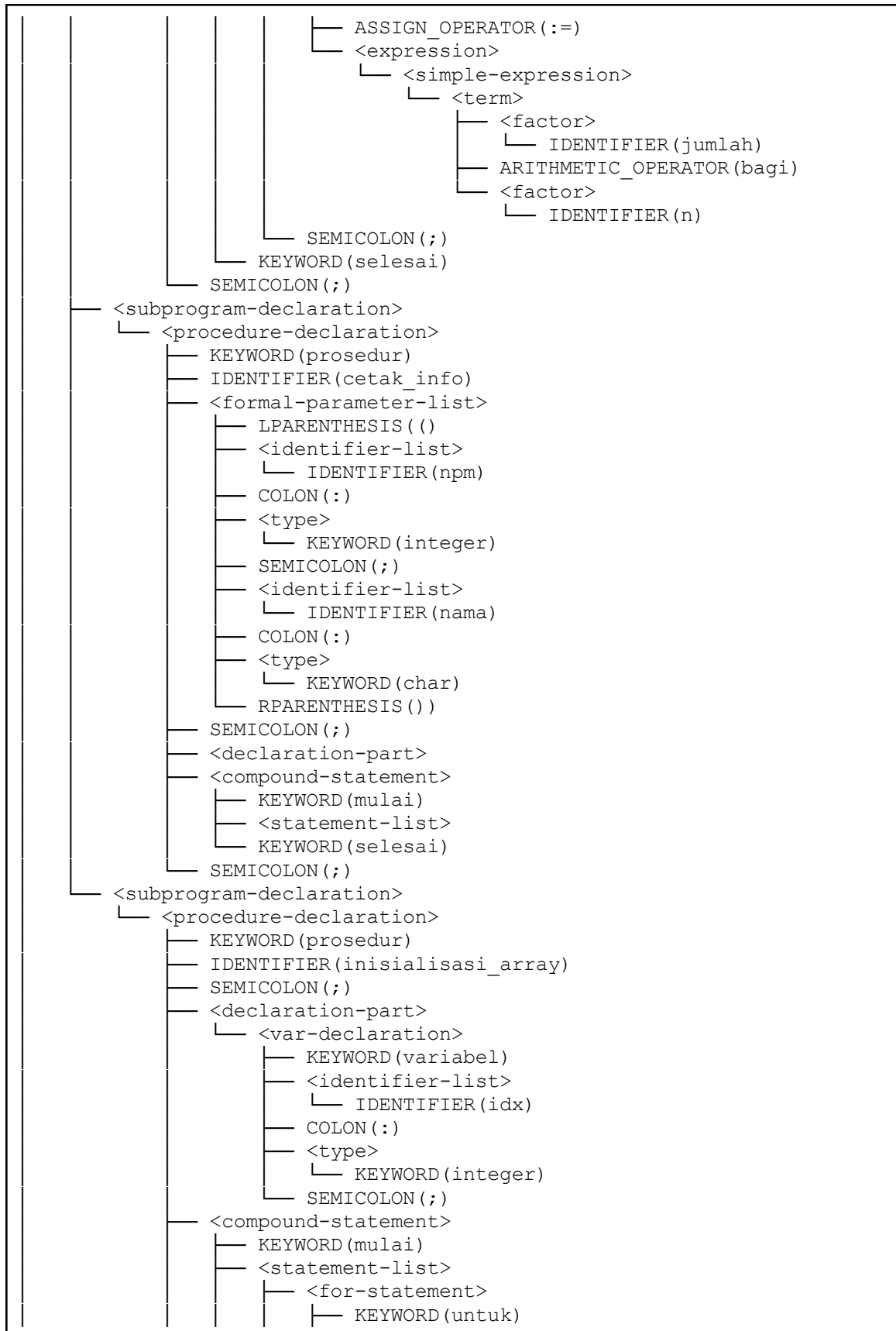
```

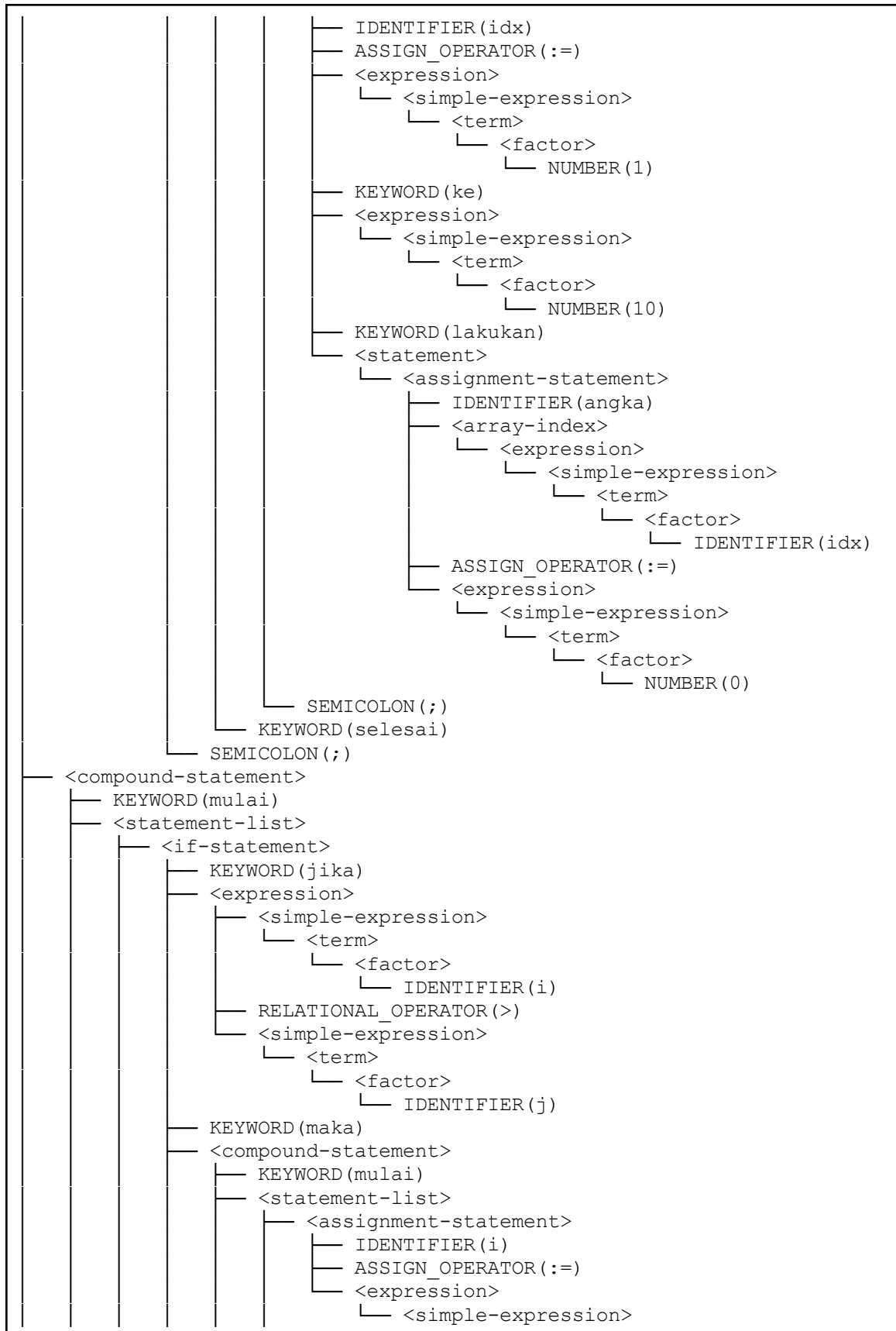
```

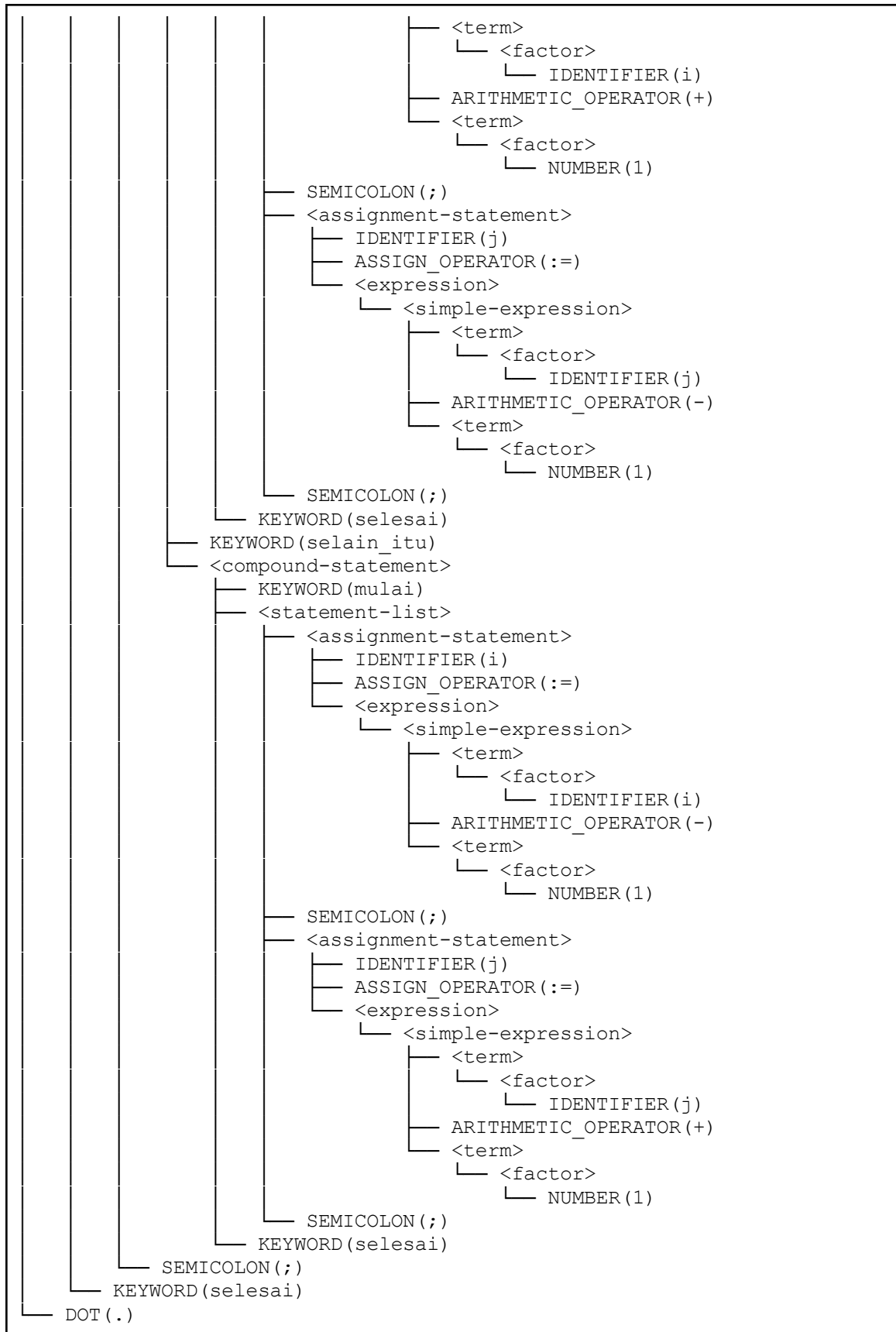
— <type>
  — KEYWORD(integer)
— SEMICOLON(;)
— <declaration-part>
— <compound-statement>
  — KEYWORD(mulai)
  — <statement-list>
    — <assignment-statement>
      — IDENTIFIER(tambah)
      — ASSIGN_OPERATOR(:=)
      — <expression>
        — <simple-expression>
          — <term>
            — <factor>
              — IDENTIFIER(a)
          — ARITHMETIC_OPERATOR(+)
          — <term>
            — <factor>
              — IDENTIFIER(b)
      — SEMICOLON(;)
    — KEYWORD(selesai)
  — SEMICOLON(;)
— <subprogram-declaration>
  — <function-declaration>
    — KEYWORD(fungsi)
    — IDENTIFIER(rata_rata)
    — <formal-parameter-list>
      — LPARENTHESIS(( )
      — <identifier-list>
        — IDENTIFIER(nilai)
      — COLON(:)
      — <type>
        — <custom-type>(NilaiUjian)
      — SEMICOLON(;)
      — <identifier-list>
        — IDENTIFIER(n)
      — COLON(:)
      — <type>
        — KEYWORD(integer)
      — RPARENTHESIS())
    — COLON(:)
    — <type>
      — KEYWORD(real)
    — SEMICOLON(;)
    — <declaration-part>
      — <var-declaration>
        — KEYWORD(variabel)
        — <identifier-list>
          — IDENTIFIER(idx)
        — COLON(:)
        — <type>
          — KEYWORD(integer)
        — SEMICOLON(;)
        — <identifier-list>
          — IDENTIFIER(jumlah)
        — COLON(:)

```









Syntax analysis completed successfully!

Bukti

```
● raif@Raif:~/YRH-Tubes-IF2224$ ./bin/compiler test/milestone-2/medium.pas
Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: test/milestone-2/medium.pas
-----
KEYWORD(program)
IDENTIFIER(TestLexerIndonesiaLengkap)
SEMICOLON(;)
KEYWORD(konstanta)
IDENTIFIER(MAX_SIZE)
RELATIONAL_OPERATOR(=)
NUMBER(100)
SEMICOLON(;)
IDENTIFIER(MIN_VALUE)
RELATIONAL_OPERATOR(=)
NUMBER(0)
```

```
ARITHMETIC_OPERATOR(+)
```

```
NUMBER(1)
```

```
SEMICOLON(;)
```

```
KEYWORD(selesai)
```

```
SEMICOLON(;)
```

```
KEYWORD(selesai)
```

```
DOT(.)
```

```
-----
Tokenization completed successfully!
```

```
Total tokens: 220
```

```
-----
Starting syntax analysis...
-----
```

```
Parse Tree:
```

```
-----
<program>
```

```
├── <program-header>
```

```
│   ├── KEYWORD(program)
```

```
│   ├── IDENTIFIER(TestLexerIndonesiaLengkap)
```

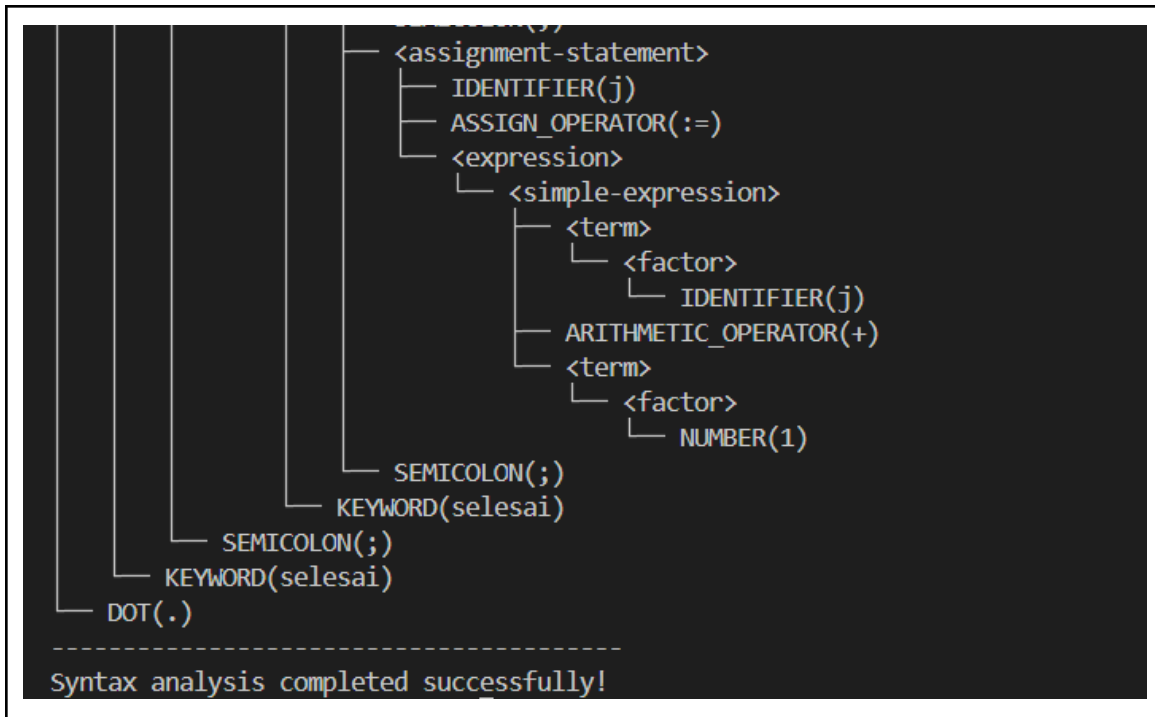
```
│   └── SEMICOLON(;)
```

```
└── <declaration-part>
```

```
    ├── <const-declaration>
```

```
        ├── KEYWORD(konstanta)
```

```
        └── IDENTIFIER(MAX_SIZE)
```



c. complete.pas

Input

```

program TestSyntaxLengkap;

{ Test case komprehensif untuk semua fitur syntax Pascal-S dengan
keyword Indonesia }

konstanta
  MAX_SISWA = 100;
  MIN_NILAI = 0;
  MAX_NILAI = 100;
  PI = 314;
  NAMA_SEKOLAH = 'SMA Negeri 1';

tipe
  NilaiRange = integer;
  DaftarNilai = larik [1..10] dari integer;
  MatriksNilai = larik [1..5] dari larik [1..5] dari real;

variabel
  i, j, k: integer;
  nilai, rata_rata, total: real;
  lulus: boolean;
  grade: char;
  nama: char;
  daftar_nilai: DaftarNilai;

```

.
.

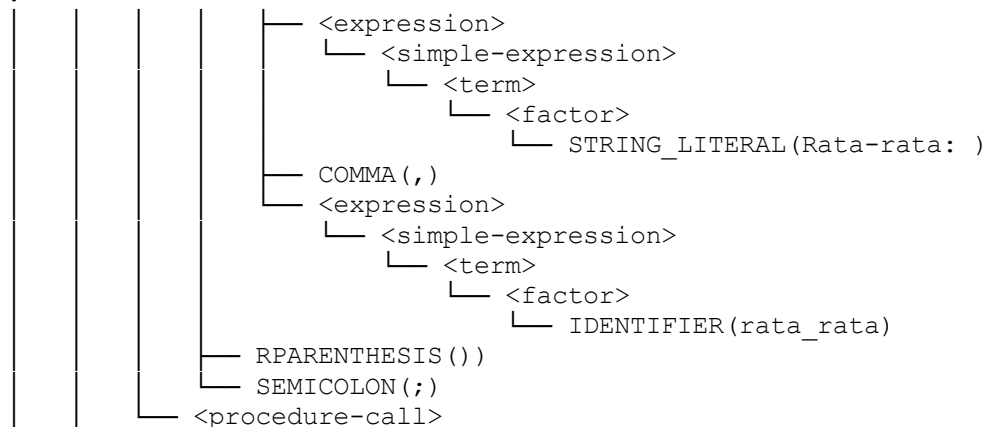
Karena teks input cukup panjang, selengkapnya dapat dilihat di:
<https://github.com/Darsua/YRH-Tubes-IF2224/blob/main/test/milestone-2/complete.pas>

Output

Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: test/milestone-2/complete.pas

```
KEYWORD(program)
IDENTIFIER(TestSyntaxLengkap)
SEMICOLON(;)
KEYWORD(konstanta)
IDENTIFIER(MAX_SISWA)
RELATIONAL_OPERATOR(=)
NUMBER(100)
SEMICOLON(;)
IDENTIFIER(MIN_NILAI)
RELATIONAL_OPERATOR(=)
NUMBER(0)
SEMICOLON(;)
IDENTIFIER(MAX_NILAI)
RELATIONAL_OPERATOR(=)
NUMBER(100)
SEMICOLON(;)
IDENTIFIER(PI)
RELATIONAL_OPERATOR(=)
NUMBER(314)
SEMICOLON(;)
IDENTIFIER(NAMA_SEKOLAH)
RELATIONAL_OPERATOR(=)
STRING_LITERAL(SMA Negeri 1)
SEMICOLON(;)
KEYWORD(tipe)
```

.
.
.



```

      IDENTIFIER(writeln)
      LPARENTHESIS((
      <parameter-list>
      <expression>
      <simple-expression>
      <term>
      <factor>
      STRING_LITERAL(Grade: )
      COMMA(,)
      <expression>
      <simple-expression>
      <term>
      <factor>
      IDENTIFIER(grade)
      RPARENTHESIS())
      SEMICOLON(;)
      KEYWORD(selesai)
      DOT(.)

```

Syntax analysis completed successfully!

Karena teks output cukup panjang, selengkapnya dapat dilihat di:
https://github.com/Darsua/YRH-Tubes-IF2224/blob/main/test/milestone-2/result/complete_result

Bukti

```

raif@Raif:~/YRH-Tubes-IF2224$ ./bin/compiler test/milestone-2/complete.pas
Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: test/milestone-2/complete.pas
-----
KEYWORD(program)
IDENTIFIER(TestSyntaxLengkap)
SEMICOLON(;)
KEYWORD(konstanta)
IDENTIFIER(MAX_SISWA)
RELATIONAL_OPERATOR(=)
NUMBER(100)
SEMICOLON(;)
IDENTIFIER(MIN_NILAI)
RELATIONAL_OPERATOR(=)
NUMBER(0)
SEMICOLON(;)
IDENTIFIER(MAX_NILAI)
RELATIONAL_OPERATOR(=)
NUMBER(100)

```

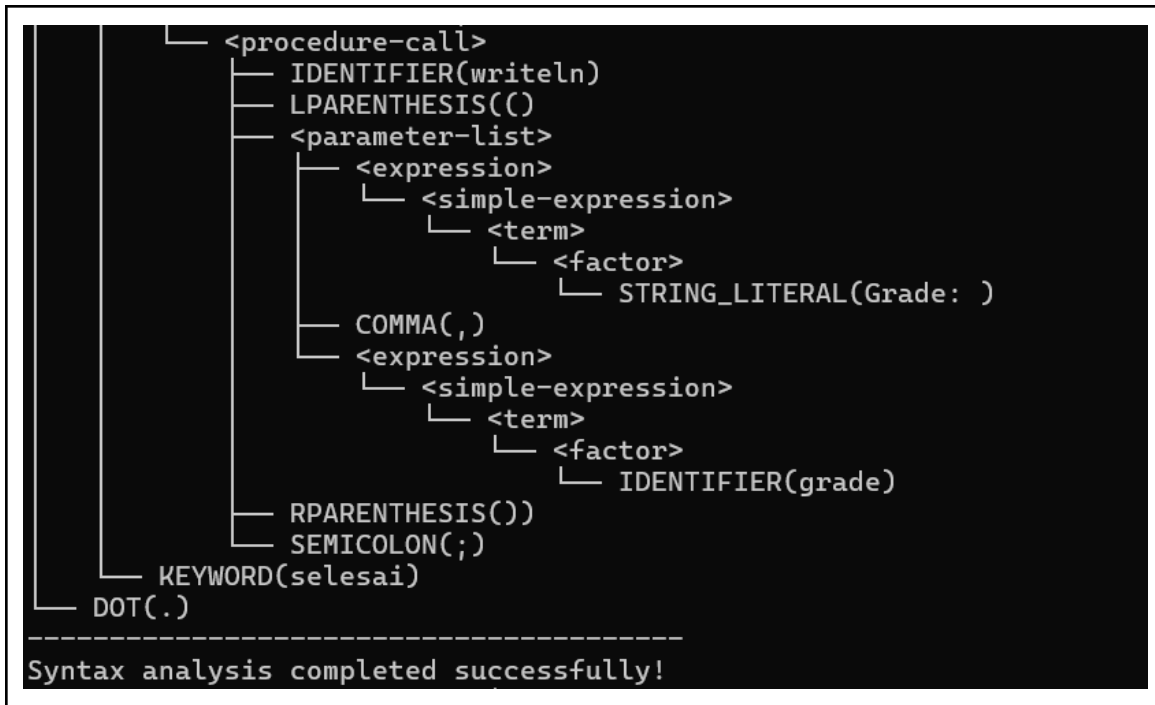
```
IDENTIFIER(grade)
RPARENTHESIS())
SEMICOLON(;)
KEYWORD(selesai)
DOT(.)
```

```
-----
Tokenization completed successfully!
Total tokens: 986
-----
```

```
Starting syntax analysis...
-----
-----
```

```
Parse Tree:
-----
```

```
<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(TestSyntaxLengkap)
│   └── SEMICOLON(;)
└── <declaration-part>
    ├── <const-declaration>
    │   ├── KEYWORD(konstanta)
    │   ├── IDENTIFIER(MAX_SISWA)
    │   └── RELATIONAL_OPERATOR(=)
```



d. hard.pas

Input
<pre> program ComprehensiveParserStressTest; { ===== COMPREHENSIVE PARSER STRESS TEST Tests all grammar rules and edge cases ===== } konstanta MAX_DEPTH = -100; MIN_VAL = 0; PI = 3.14159; E = 2.71828; TOLERANCE = 0.001; EMPTY_STR = ''; GREETING = 'Hello, Parser!'; tipe ScoreRange = integer; Matrix = larik [1..10] dari larik [1..10] dari real; Vector = larik [0..99] dari integer; NestedArray = larik [1..5] dari larik [1..5] dari larik [1..5] dari integer; variabel i, j, k, temp, result: integer; x, y, z, sum, avg: real; flag, done, valid, ready: boolean; </pre>

```
ch, grade: char;
scores: Vector;
mat: Matrix;
depth: NestedArray;
```

.
.
.

Karena teks input cukup panjang, selengkapnya dapat dilihat di:
<https://github.com/Darsua/YRH-Tubes-IF2224/blob/main/test/milestone-2/hard.pas>

Output

Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: test/milestone-2/hard.pas

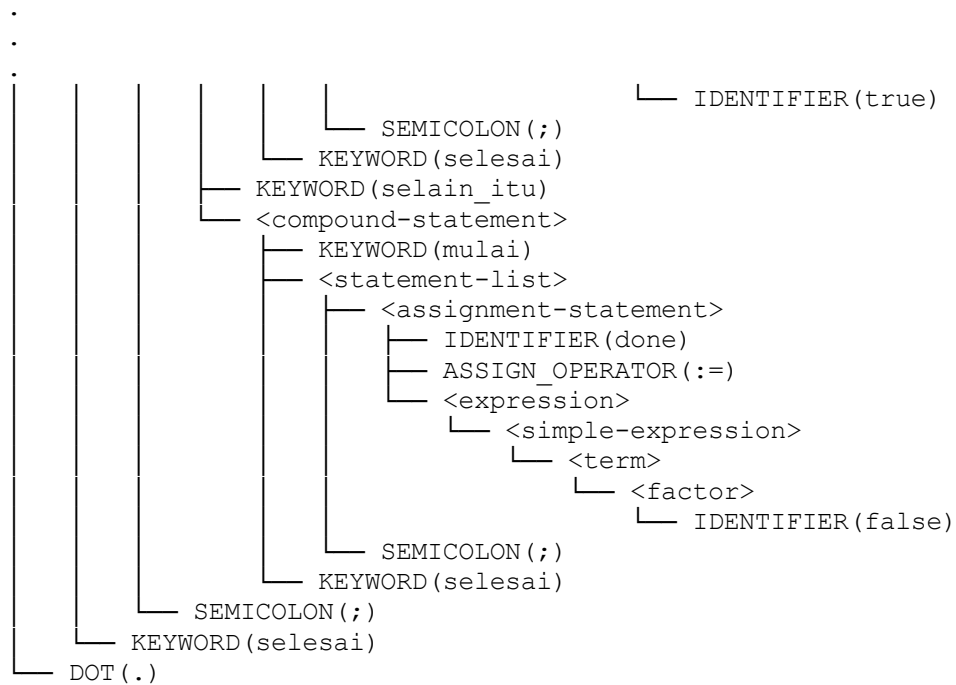
```
-----
KEYWORD(program)
IDENTIFIER(ComprehensiveParserStressTest)
SEMICOLON(;)
KEYWORD(konstanta)
IDENTIFIER(MAX_DEPTH)
RELATIONAL_OPERATOR(=)
ARITHMETIC_OPERATOR(-)
NUMBER(100)
SEMICOLON(;)
IDENTIFIER(MIN_VAL)
RELATIONAL_OPERATOR(=)
NUMBER(0)
.
.
.
SEMICOLON(;)
KEYWORD(selesai)
SEMICOLON(;)
KEYWORD(selesai)
DOT(.)
-----
```

Tokenization completed successfully!
Total tokens: 1689

Starting syntax analysis...

Parse Tree:

```
-----
<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(ComprehensiveParserStressTest)
│   └── SEMICOLON(;)
└── <declaration-part>
    ├── <const-declaration>
    │   └── KEYWORD(konstanta)
```

Syntax analysis completed successfully!

Karena teks output cukup panjang, selengkapnya dapat dilihat di:

https://github.com/Darsua/YRH-Tubes-IF2224/blob/main/test/milestone-2/result/hard_result

Bukti

```

raif@Raif:~/YRH-Tubes-IF2224$ ./bin/compiler test/milestone-2/hard.pas
Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: test/milestone-2/hard.pas
-----
KEYWORD(program)
IDENTIFIER(ComprehensiveParserStressTest)
SEMICOLON(;)
KEYWORD(konstanta)
IDENTIFIER(MAX_DEPTH)
RELATIONAL_OPERATOR(=)
ARITHMETIC_OPERATOR(-)
NUMBER(100)
SEMICOLON(;)
IDENTIFIER(MIN_VAL)
RELATIONAL_OPERATOR(=)
NUMBER(0)

```

```
IDENTIFIER(false)
SEMICOLON(;)
KEYWORD(selesai)
SEMICOLON(;)
KEYWORD(selesai)
DOT(.)
```

```
-----
Tokenization completed successfully!
Total tokens: 1689
-----
```

```
Starting syntax analysis...
-----
```

```
Parse Tree:
-----
```

```
<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(ComprehensiveParserStressTest)
│   └── SEMICOLON(;)
└── <declaration-part>
    ├── <const-declaration>
    │   ├── KEYWORD(konstanta)
    │   ├── IDENTIFIER(MAX_DEPTH)
    │   ├── RELATIONAL_OPERATOR(=)
    │   ├── <constant-value>(-100)
    │   └── SEMICOLON(;)
    └──
```

```

    └── SEMICOLON(;)
    └── KEYWORD(selesai)
    └── KEYWORD(selain_itu)
    └── <compound-statement>
        ├── KEYWORD(mulai)
        ├── <statement-list>
        │   ├── <assignment-statement>
        │   │   ├── IDENTIFIER(done)
        │   │   ├── ASSIGN_OPERATOR(:=)
        │   │   └── <expression>
        │   │       ├── <simple-expression>
        │   │       │   ├── <term>
        │   │       │   │   ├── <factor>
        │   │       │   │   └── IDENTIFIER(false)
        │   │       └── SEMICOLON(;)
        │   └── KEYWORD(selesai)
        └── SEMICOLON(;)
    └── KEYWORD(selesai)
    └── DOT(.)
```

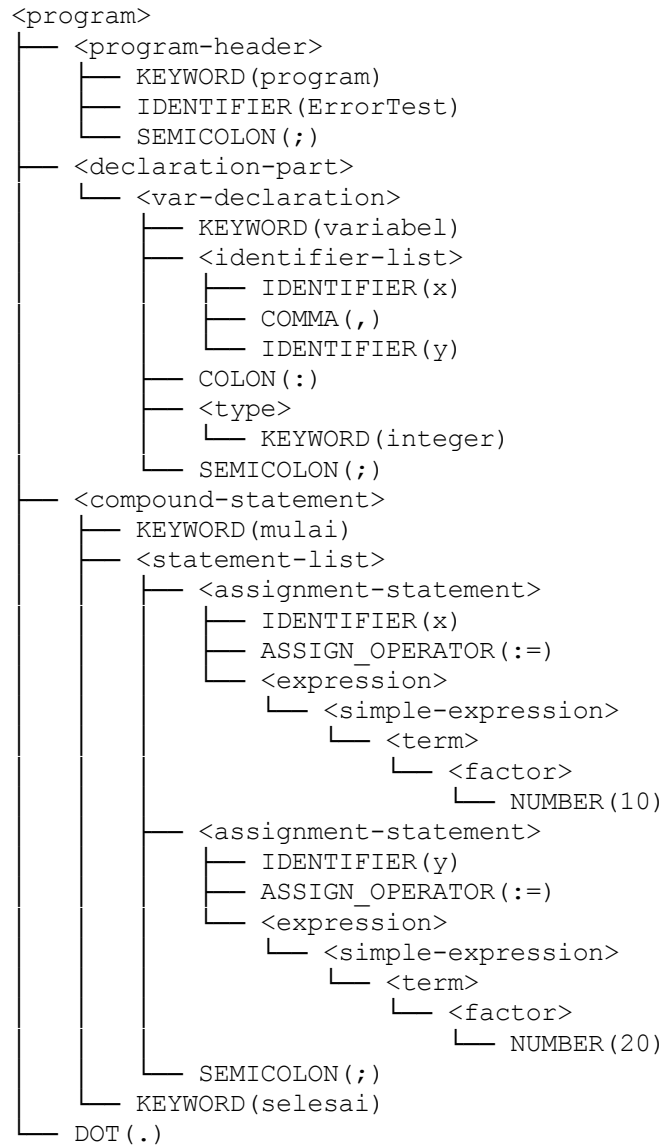
```
-----
Syntax analysis completed successfully!
raif@Raif:~/YRH-Tubes-IF2224$
```

e. **error_test.pas**

Input
<pre>program ErrorTest; variabel x, y: integer; mulai x := 10 y := 20; selesai.</pre>
Output
<pre>./bin/compiler test/milestone-2/error_test.pas Using DFA-based lexer DFA rules file: rules/pascal_lexicon.dfa Processing file: test/milestone-2/error_test.pas ----- KEYWORD(program) IDENTIFIER(ErrorTest) SEMICOLON(;) KEYWORD(variabel) IDENTIFIER(x) COMMA(,) IDENTIFIER(y) COLON(:) KEYWORD(integer) SEMICOLON(;) KEYWORD(mulai) IDENTIFIER(x) ASSIGN_OPERATOR(:=) NUMBER(10) IDENTIFIER(y) ASSIGN_OPERATOR(:=) NUMBER(20) SEMICOLON(;) KEYWORD(selesai) DOT(.) ----- Tokenization completed successfully! Total tokens: 20 ----- Starting syntax analysis... ----- ===== SYNTAX ERROR ===== Position: Token #15 of 20 Message: Expected ';' after statement Current: IDENTIFIER(y) Previous: NUMBER(10)</pre>

Next: ASSIGN_OPERATOR (:=)

Parse Tree:



Syntax analysis completed successfully!

Bukti

```

raif@Raif:~/YRH-Tubes-IF2224$ ./bin/compiler test/milestone-2/error_test.pas
Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: test/milestone-2/error_test.pas
-----
KEYWORD(program)
IDENTIFIER(ErrorTest)
SEMICOLON(;)
KEYWORD(variabel)
IDENTIFIER(x)
COMMA(,)
IDENTIFIER(y)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
KEYWORD(mulai)
IDENTIFIER(x)
ASSIGN_OPERATOR(:=)
NUMBER(10)
IDENTIFIER(y)
ASSIGN_OPERATOR(:=)
NUMBER(20)
SEMICOLON(;)
KEYWORD(selesai)
DOT(.)
-----
Tokenization completed successfully!
Total tokens: 20
-----
Starting syntax analysis...
-----

=====
SYNTAX ERROR
=====
Position: Token #15 of 20
Message: Expected ';' after statement
Current: IDENTIFIER(y)
Previous: NUMBER(10)
Next: ASSIGN_OPERATOR(:=)
=====

```

Parse Tree:

```
<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(ErrorTest)
│   └── SEMICOLON(;)
├── <declaration-part>
│   └── <var-declaration>
│       ├── KEYWORD(variabel)
│       ├── <identifier-list>
│       │   ├── IDENTIFIER(x)
│       │   ├── COMMA(,)
│       │   └── IDENTIFIER(y)
│       ├── COLON(:)
│       ├── <type>
│       │   └── KEYWORD(integer)
│       └── SEMICOLON(;)
├── <compound-statement>
│   ├── KEYWORD(mulai)
│   ├── <statement-list>
│   │   ├── <assignment-statement>
│   │   │   ├── IDENTIFIER(x)
│   │   │   ├── ASSIGN_OPERATOR(:=)
│   │   │   └── <expression>
│   │   │       └── <simple-expression>
│   │   │           └── <term>
│   │   │               └── <factor>
│   │   │                   └── NUMBER(10)
│   │   ├── <assignment-statement>
│   │   │   ├── IDENTIFIER(y)
│   │   │   ├── ASSIGN_OPERATOR(:=)
│   │   │   └── <expression>
│   │   │       └── <simple-expression>
│   │   │           └── <term>
│   │   │               └── <factor>
│   │   │                   └── NUMBER(20)
│   │   └── SEMICOLON(;)
│   └── KEYWORD(selesai)
└── DOT(.)
```

Syntax analysis completed successfully!

4. Kesimpulan & Saran

a. Kesimpulan

Implementasi syntax analyzer untuk Milestone 1 Tugas Besar IF3170 telah berhasil diselesaikan dengan menerapkan konsep Recursive Descent Parsing. Parser yang dibangun mampu menjalankan fungsinya, yaitu menghasilkan sebuah parse tree dari program Pascal-S masukan menggunakan aturan-aturan produksi yang ada.

Berdasarkan hasil pengujian terhadap lima file uji (dasar.pas, medium.pas, complete.pas, hard.pas, dan error_test.pas), parser telah mampu untuk:

- mengenali dan memproses semua struktur deklarasi Pascal-S dengan keyword bahasa Indonesia,
- memproses semua atau sebagian besar jenis statement yang didukung oleh grammar Pascal-S,
- menangani ekspresi kompleks dengan berbagai operator (baik aritmatika, relasional, logika, unary, atau ekspresi bersarang dengan parentheses hingga multiple levels),
- membangun serta menampilkan parse tree yang terstruktur,
- memberikan error handling yang informatif dengan menampilkan posisi token yang menyebabkan error dan konteksnya,
- serta mendukung penanganan bilangan real, bilangan negatif, deklarasi subrange, akses record, nested procedure, dan lain-lain.

Secara keseluruhan, parser yang diimplementasikan telah fungsional dan memenuhi spesifikasi yang dibutuhkan untuk milestone ini. Parser ini dapat digunakan lebih lanjut ke tahap-tahap selanjutnya dalam mengonstruksi sebuah compiler.

b. Saran

Parser sudah dapat bekerja dengan baik. Namun, ada beberapa penambahan atau perbaikan yang dapat diimplementasikan. Misalnya untuk debugging yang lebih baik dapat ditambahkan informasi baris kode yang mengalami error seperti pada SQL atau Python agar lebih jelas bagian mana yang menyebabkan error. Selain itu, dapat pula ditambahkan debug mode supaya pengguna dapat melihat langsung proses parsing atau berbagai mode lain.

Lampiran

- Link Repository Github:
[Darsua/YRH-Tubes-IF2224: NieR: Finite Automata](#)
- Grammar yang digunakan pada syntax analysis di program ini diimplementasikan secara hard-coded. Grammarsnya adalah sebagai berikut.
Grammar ditulis dengan format BNF yang pernah diajarkan di kelas melalui PPT dosen:

Notasi BNF (Backus-Nour Form)

- Aturan Produksi bisa dinyatakan dengan notasi BNF
- BNF menggunakan abstraksi untuk struktur syntax

$::=$	sama identik dengan simbol $\rightarrow$
$ $	sama dengan atau
$\langle \rangle$	pengapit simbol non terminal
$\{ \}$	Pengulangan dari 0 sampai n kali

Misalkan aturan produksi sbb:

$$E \rightarrow T \mid T+E \mid T-E$$
$$T \rightarrow a$$

Notasi BNFnya adalah

$$E ::= \langle T \rangle \mid \langle T \rangle + \langle E \rangle \mid \langle T \rangle - \langle E \rangle$$
$$T ::= a$$

```
<program> ::= <program-header> <declaration-part> <compound-statement> DOT
<program-header> ::= KEYWORD(program) IDENTIFIER SEMICOLON
<declaration-part> ::= {<const-declaration>} {<type-declaration>}
{<var-declaration>} {<subprogram-declaration>}
<const-declaration> ::= KEYWORD(konstanta) {IDENTIFIER = value SEMICOLON}
<type-declaration> ::= KEYWORD(tipe) {<type-definition>}
<type-definition> ::= IDENTIFIER RELATIONAL_OPERATOR(=) <type>
SEMICOLON
<var-declaration> ::= KEYWORD(variabel) {<identifier-list> COLON <type>
SEMICOLON}
<identifier-list> ::= IDENTIFIER {COMMA + IDENTIFIER}
<type> ::= KEYWORD(integer/real/boolean/char) | <array-type> | <range> |
<record-type> | IDENTIFIER
<array-type> ::= KEYWORD(larik) LBRACKET <range> RBRACKET
KEYWORD(dari) <type>
<record-type> ::= KEYWORD(rekaman) {<identifier-list> COLON <type>
```


SEMICOLON} KEYWORD(selesai)
<range> ::= <expression> RANGE_OPERATOR(..) <expression>
<subprogram-declaration> ::= <procedure-declaration> | <function-declaration>
<procedure-declaration> ::= KEYWORD(prosedur) IDENTIFIER
 <formal-parameter-list> SEMICOLON <declaration-part> <compound-statement>
 SEMICOLON | KEYWORD(prosedur) IDENTIFIER SEMICOLON <declaration-part>
 <compound-statement> SEMICOLON
<function-declaration> ::= KEYWORD(fungsi) IDENTIFIER <formal-parameter-list>
 COLON <type> SEMICOLON <declaration-part> <compound-statement>
 SEMICOLON | KEYWORD(fungsi) IDENTIFIER COLON <type> SEMICOLON
 <declaration-part> <compound-statement> SEMICOLON
<formal-parameter-list> ::= LPARENTHESIS <identifier-list> COLON <type>
 RPARENTHESIS | LPARENTHESIS <identifier-list> COLON <type> {SEMICOLON
 <identifier-list> COLON <type>} RPARENTHESIS
<compound-statement> ::= KEYWORD(mulai) <statement-list> KEYWORD(selesai)
<statement-list> ::= <assignment-statement> | <if-statement> | <while-statement> |
 <for-statement> | <procedure-call> | <compound-statement> {SEMICOLON
 <assignment-statement> | <if-statement> | <while-statement> | <for-statement> |
 <procedure-call> | <compound-statement>}
<assignment-statement> ::= IDENTIFIER {DOT IDENTIFIER | LBRACKET
 <expression> RBRACKET} ASSIGN_OPERATOR <expression>
<if-statement> ::= KEYWORD(jika) <expression> KEYWORD(maka)
 <statement_or_call> KEYWORD(selain_itu) <statement_or_call>
<statement_or_call> ::= <assignment-statement> | <procedure-call> |
 <compound-statement>
<while-statement> ::= KEYWORD(selama) <expression> KEYWORD(lakukan)
 <statement_or_call>
<for-statement> ::= KEYWORD(untuk) IDENTIFIER ASSIGN_OPERATOR
 <expression> (KEYWORD(ke) | KEYWORD(turun_ke)) <expression>
 KEYWORD(lakukan) <statement_or_call>
<procedure/function-call> ::= IDENTIFIER LPARENTHESIS <parameter-list>
 RPARENTHESIS SEMICOLON
<parameter-list> ::= LPARENTHESIS <expression> RPARENTHESIS |
 LPARENTHESIS <expression> {COMMA <expression>} RPARENTHESIS
<expression> ::= <simple-expression> | <simple-expression>
 RELATIONAL_OPERATOR <simple-expression>
<simple-expression> ::= <term> {<additive-operator> <term>} |
 ARITHMETIC_OPERATOR(+/-) <term> {<additive-operator> <term>}
<term> ::= <factor> {<multiplicative-operator> <factor>}
<factor> ::= IDENTIFIER {DOT IDENTIFIER | LBRACKET <expression>
 RBRACKET | LPARENTHESIS <parameter-list> RPARENTHESIS} | NUMBER |
 STRING_LITERAL | CHAR_LITERAL | LPARENTHESIS <expression>
 RPARENTHESIS | LOGICAL_OPERATOR(tidak) <factor> |
 ARITHMETIC_OPERATOR(+|-) <factor>
<relational-operator> ::= = | <> | < | <= | > | >=
<additive-operator> ::= + | - | atau

<multiplicative-operator> ::= * / bagi mod dan
--

c. Pembagian Tugas

Anggota	Tugas	Kontribusi
Muhammad Ra'if Alkautsar (13523011)	Parser	35%
	Laporan	20%
	Main	50%
Fajar Kurniawan (13523027)	Parser	20%
	Laporan	40%
Darrel Adinarya Sunanda (13523061)	Laporan	10%
	Main	50%
Reza Ahmad Syarif (13523119)	Parser	45%
	Laporan	30%

Referensi

“PASCAL-S: A Subset and its implementation”

<http://pascal.hansotten.com/uploads/pascals/PASCAL-S%20A%20subset%20and%20its%20Implementation%20012.pdf>

“Let’s Build A Simple Interpreter. Part 7: Abstract Syntax Trees”

<https://ruslanspivak.com/lbasi-part7>

“Contoh Gambar Parse Trees”

https://textx.github.io/Arpeggio/latest/images/calc_parse_tree.dot.png